good practices

###
[code](https://code.tutsplus.com/tutorials/top-15-best-practices-for-writing-super-readable-code--net-8118)

- Commenting and Documentation
- Consistent Indentation
- Avoid Obvious Comments
- Code Grouping
- Consistent Naming Scheme
  - camelCase
  - under_scores
- [**DRY**](https://thefullstack.xyz/dry-yagni-kiss-tdd-soc-bdfu) (Don't Repeat Yourself) or [**DIE**](https://thefullstack.xyz/dry-yagni-kiss-tdd-soc-bdfu) (Duplication is Evil) principle
- [**KISS**](https://thefullstack.xyz/dry-yagni-kiss-tdd-soc-bdfu) (Keep It Simple Stupid)
- [**YAGNI**](https://thefullstack.xyz/dry-yagni-kiss-tdd-soc-bdfu) (You Aren’t Gonna Need It)
- [**SOC**](https://thefullstack.xyz/dry-yagni-kiss-tdd-soc-bdfu) (Separation of Concerns)
- Avoid Deep Nesting
- Handling errors
- Limit Line Length
- File and Folder Organization
- Consistent Temporary Names
- Separation of Code and Data
- Code Refactoring

Go

- [**Go**](https://golang.org/doc/effective_go.html)
- [**gofmt**](https://golang.org/cmd/gofmt/)
- [**goimports**](https://godoc.org/golang.org/x/tools/cmd/goimports)
- [**foimports vs gofmt**](https://goinbigdata.com/goimports-vs-gofmt/)

CSS

- [**CSS**](https://www.tothenew.com/blog/10-best-practices-in-css/)

Dockerfile

-
[**Dockerfile**](https://docs.docker.com/develop/develop-images/dockerfile_best-practices/)

Time Limitation

- Every computing programs should execute in a time limit of 5 minutes.

Game performance

Performance is essential, so that's why you have to aim for less than 16.7ms(1000ms/60f), because in 16.7ms the browsers job and your work must be completed in each frame.

Use
[`requestAnimationFrame`](https://developer.mozilla.org/en-US/docs/Web/API/window/requestAnimationFrame) to sync your changes

- You can try to reuse memory to avoid jank when the garbage collector pass

Jank animation can be caused by loading too much information, for instance:

- **JavaScript**
- **Styles** that considers which style apply to which element
- **Layout** that calculates the geometry of the pages (example: recalculating the width and height of a page)
- **Painting**, normally when layout is triggered we must repaint. Repainting an element every time it animates
- **Compositing** that consists on placing the pages together at the end

In order to improve performance we must remove the causes above, that are more costly: layout and painting.

The best way to remove layout and painting is to use
[`transform`](https://developer.mozilla.org/en-US/docs/Web/CSS/transform) and
[`opacity`](https://developer.mozilla.org/en-US/docs/Web/CSS/opacity)

Removing layout can be done using transform:

```js
// bad
// this will trigger the layout to recalculate everything and repaint it again
box.style.left = `${x * 100}px`

// good
// this way its possible to lose the layout
box.style.transform = `translateX(${x * 100}px)`
```

It is possible to remove painting by adding a layer:

```css
/* this will take care of the painting by creating a layer and transform it*/
  #box {
    width: 100px;
    height: 100px;
    ....
    will-change: transform;
  }
```

```

By creating a new layer you can remove painting, but "there is always a tradeoff". If we add to much layers it will increase the **composition** and **update tree**. In conclusion you must promote a new layer only if you know you are going to use it. Performance is the key to a good animation. "Performance is the art of avoiding work".

### UI/UX

- [Best Practices](https://www.uxpin.com/studio/blog/guide-design-consistency-best-practices-ui-ux-designers/)
```